**NATURAL LANGUAGE PROCESSING**

Project Title

# HealthGuard: Auditing Bias and Explainability in Health Claim Classification

| | |
|---|---|
| **Programme** | BBA (Artificial Intelligence) |
| **Academic Year** | 2026-27 |
| **Team Members** | 1. [Anshika Choubey] - [25030422017] |
| **Faculty** | Dr. Nabanita Choudhury, Assistant Professor |


**SYMBIOSIS ARTIFICIAL INTELLIGENCE INSTITUTE (SAII)**

Constituent of Symbiosis International (Deemed University)
Lavale, Pune - 412115

## Report Sections

## 1. Abstract

Health claims proliferate rapidly, and most go unreviewed by professionals. Automated classifiers are valuable tools, and can help by highlighting those claims already judged false by fact-checkers, so that they may be prioritized for review. This report presents HealthGuard, a binary text classifier for health claims, trained on the PUBHEALTH fact-checking dataset. Only claims labelled True or False are retained; each is converted into TF-IDF features of single words and two-word phrases, and a Logistic Regression model trained with balanced class weights is applied. On the held-out test set of 987 claims, the model achieves 76.3% accuracy and macro-F1 of 0.757.These figures are satisfactory, but conceal variation: on partitioning the test set by length, it becomes clear that shorter claims (15 words or fewer) are classified with 79.8% accuracy, but longer claims achieve only 67.0%. The discrepancy is widest on verified claims: only 19.8% of long True claims are labelled True, compared to 84.4% of short ones. The model also shows strong bias towards very common words, such as "the", "is" and "and", and so the phrasing of a claim can be as important as its contents. For practicality, the trained model is deployed within a small web application. A user submits a claim, and receives a label, confidence score, and a brief rationale highlighting words most responsible for the decision. The application is a proof-of-concept; it is not a medical tool, and the report considers the dangers of conflating an automated label with a verdict.

## 2. Introduction

### 2.1 Background and context

Automated health fact-checking (colloquially, virus-checking)

Public trust in health information is becoming a practical problem. People are searching for treatments, reading about vaccines and sharing information about diseases in social media; a large portion of this information has never been checked by anyone qualified to do so. Professional fact-checkers review some of the most popular claims, but these are a tiny proportion of the claims overall; journalists, platform moderators and public-health teams all find themselves in the same situation: a claim has to be read, assessed and interpreted before it can be addressed

Automated fact-checking is an attempt to loosen this bottleneck: ideally, not to replace a human reviewer, but to help them decide where to spend their time. A system that ranks suspicious claims by their probable falsehood, or a claim that is similar to a previously debunked one, can greatly reduce the amount of time spent. The difficulty is that a classifier that is seemingly confident may also be wrong in ways that are hard to evaluate; for a health claim, a mistake may mean the difference between a patient ignoring potentially helpful advice or embracing something that will cause them harm.

This project works at the bottom end of this spectrum. It does not attempt to determine whether a claim is medically true, because that requires reasoning over evidence that the model will never have access to. This project asks a narrower question: given the text of a health claim that has already been labelled by a fact-checker, can a simple, readable model be trained to predict that label, and where does it fail? The answer will depend both on the model's construction and on its evaluation.

### 2.2 Related work

Several research groups have investigated automated fact-checking. Vlachos and Riedel [8] defined the task and assembled one of the early data sets of claims with associated verdicts. The FEVER project [9] built a large benchmark in which claims are checked against Wikipedia evidence, showing how much the evidence-retrieval step contributes. PUBHEALTH [1] is closest to this project in that it collects health claims from fact-checking sites, labels them with the verdict of the fact-checker, and includes an explanatory text by the fact-checker. Kotonya and Toni used it to investigate explainable fact-checking, which is the same motivation as this project's explanations.

Most recent research in text classification uses large pretrained language models such as BERT [10]. These tend to achieve higher accuracy than classical methods, but the internal representation is dense and not human-readable. An explanation of a single prediction typically requires a separate technique such as LIME [11] that approximates the model locally with a simpler one. Other authors have pointed out that classifiers can encode biases that are only visible when accuracy is measured on subgroups. Buolamwini and Gebru [12] illustrate the problem clearly on the face recognition task, and the same considerations apply to text, with the subgroups possibly being claim length, style or topic.

Model documentation is another related area. Mitchell and colleagues [13] proposed model cards: short documents that describe what a model is used for, how it was evaluated, and where it should not be used. This project's Annexure A follows this idea in that it compiles the same information in a succinct format.

The work described here is closest in spirit to the subgroup audit and the explainability investigations. It uses a classical pipeline on purpose. A TF-IDF representation plus a linear classifier is a weak baseline by today's standards, but it has the advantage that every decision can be traced back to individual words. For a project that is concerned with understanding where a classifier fails, this may be a valuable trade-off for the lost accuracy.

### 2.3 Motivation for choosing the problem

I chose this problem for three reasons. The first is that health claims are high-value texts, so the cost of wrong classification is significantly higher than in many other classification problems. The second reason is that the dataset is public, annotated by professionals, and openly available, which adds to the credibility of the results. Finally, the question of fairness and bias is of great importance, since unfair algorithms can expose a person to actions that they would not have otherwise suffered. If, for example, the model treats short claims differently from long ones, or claims written in a particular style differently from others, this can lead to unfair outcomes for the user of the model, without anyone being aware of it. Therefore, evaluating the fairness of the model is a central aspect of this project, and not a mere exercise.

### 2.4 Role of NLP in the proposed solution

NLP is necessary to the process since the input is free text while the output is for the classifier, who takes numbers. The representation stage defines what the classifier sees. The project uses a representation where each claim is a sparse vector, with a dimension for every word or words pair in the training vocabulary, weighted according to their TF-IDF. Then, the classifier is trained to have one weight per dimension. As the representation is interpretable, the weights can be read out as

statements about words, allowing the explanation step. Thus, the NLP task is to choose a representation that can be the basis for an explanation.

### 2.5 Structure of the report

Section 3 provides the problem and objectives statement and Section 4 describes the dataset, while Section 5 explains the preprocessing choices. Section 6 covers the feature representation and Section 7 the overall method, while Section 8 is dedicated to the model. The implementation and the resulting web application are described in Section 9. The results are presented in Section 10, the analysis of the failures is in Section 11, and we discuss the ethical concerns and limitations in Section 12. The final concluding remarks and future work directions are given in Section 13, followed by the reference list (Section 14), the repository link, the contribution statement and the annexure with the model card.

### 2.6 How this report is organised around the reader

The report is meant for two kinds of people. The first one who is an examiner who wishes to see whether the project meets its requirements of the course, makes suitable choices of method, and makes conclusions which follow from the analysis. The second one is a student or practitioner who might wish to rebuild the work or extend it to another dataset. For this reason, the report explains the reasons behind every decision as much as the outcome of the decision, and indicates where a result is present rather than simply stating a figure without context.

I have tried to keep the technical sections readable without losing precision. Equations are used only where they are necessary to explain a mechanism which would be otherwise be unclear, like the TF-IDF weighting and the logistic regression. Every number in the results section can be traced to the notebook output or a stored file in the repository, and the probe examples can be reproduced by anyone who loads the saved model.

### 2.7 Contribution of this work

The contribution of the project is small in scope but specific in nature. It demonstrates that a transparent baseline trained on publicly available health fact-checking data achieves a respectable headline accuracy while performing badly on one identifiable subgroup of inputs. It demonstrates that the same baseline relies on function words in a manner unexpected by the reader. And it demonstrates how such findings can be made visible to the user by an interface that displays both the prediction and the explanations for it.

• A reproducible pipeline for binary health-claim classification on PUBHEALTH with its filtering decisions documented.

• A length-based subgroup audit with precision, recall and macro-F1 reported for each group.

• An exact, per-claim explanation based on the linear model's contributions together with a working web application that presents it.

• A model card that states the intended use, the limitations and the recommended next steps.

## 3. Problem Statement and Objectives

### 3.1 Problem statement

Readers are faced with an increasing number of health claims that have not been subjected to verification. Automatic classifiers can help identify statements that fact-checkers have declared false, but their ability to do so could vary depending on the formulation of the claim. The purpose of this paper is to create a TF-IDF and Logistic Regression classifier based on the public health fact-checking dataset and evaluate its performance on both short and long claims in order to determine its reliability, bias, and explainability.

In other words, there are three questions that this study aims to address. First, is it possible to train an easy-to-express model that can distinguish between true and false health claims with sufficient accuracy? Second, will this accuracy be high enough regardless of the length of the claims included? Finally, can the reader understand the logic of classifying sentences put forward by the model?

### 3.2 Research questions

Research questions

RQ1. What is the accuracy rate of the TF-IDF and Logistic Regression algorithm for classifying health claims as true or false?

RQ2. Are there differences in accuracy rates in classifying short- (≤15 words) and long (>15 words) claims?

RQ3. What words or phrases does the algorithm use as predictors, and do they predict a claim’s truth or lie based on meaning or merely based on words?

### 3.3 Objectives

Objective 1: create a TF-IDF and Logistic Regression pipeline on the PUBHEALTH dataset.

Objective 2: audit performance on separate claims of 15_words_or_fewer and claims of _more_than_15_words.

Objective 3: extract the largest coefficients to help explain the predictions.

Objective 4: deliver a notebook that reproduces the results, and a working website that uses the trained model.

### 3.4 Scope and assumptions

The project is restricted to English claims which are labeled True or False by the dataset. Mixture and unproven claims were excluded since that type of data does not allow for a simple binary choice. The project does not attempt to fact-check the claims using medical literature or modern evidence. The output from the model is considered a prediction of the fact-checker label and not the actual truth of the claim. These restrictions are further detailed in Section 12

### 3.5 Success criteria

Before doing the experiments, I set three criteria that the model should meet. First one is that the model needs to show that it significantly outperforms a unimportant baseline on the test set. Second one is that the length audit needs to be made public, even if the results are not positive towards the model. Finally, every prediction that the application makes needs to have an explanation that shows how the model arrives at the result, rather than simply citing a generic statement. The results in Section 10 are evaluated against these criteria.

### 3.6 Why the audit matters for this problem

An audit is a method of questioning a classifier about its headline figure. One accuracy figure represents the idea that all kinds of claim are equally likely to be presented and handled well, but the audit directly examines that assumption, breaking the test set into groups and reporting the same metrics per group. Thus, if the groups differ in their performance, then the headline number is an average over potentially very different outcomes, and users need to be aware of what group they fall into.

This is particularly important for health claims, where there are both short, headline claims and long, carefully hedged claims. If someone reading a long claim is the one most needing a reliable answer, then a model that does worst on this kind of input fails those who need it most. The audit was designed to answer the question that the stakeholders of such a system would ask first: does the system work for the claims I am likely to see?

## 4. Dataset Description

### 4.1 Source

The project makes use of PUBHEALTH [1], a publicly available set of health claims collected from fact-checking websites. Each claim has been assessed by a fact-checker, who assigned a verdict and provided an explanation. The data is also available on the Hugging Face datasets hub [2], and I used the hub's listing to determine the label set and the split sizes. The notebook itself loads the tab-separated PUBHEALTH files, since they contain the same records in a format the original authors published.

The data is distributed in train, validation and test splits. The splits were used as provided, and I did not make my own partitions, so that the results can be compared with other work that uses the same files.

### 4.2 Fields and their meaning

Each record contains the claim identifier, the claim text, the publication date, the fact-checker's explanation, the name of the fact-checker, the main text of the source article, the list of source links, the label, and the subject tags. The last one is the most informative, as it represents the label of each instance. In the initial set, the label can take five types of values: true, false, mixture, unproven, and a small amount of missing instances with no labels.

For our purposes, only two fields are of interest, namely the text of the claim (input) and the label (output). Other fields, namely the explanation and the text of the article, contain valuable information, but using them would change the nature of the task to evidence-based claim verification. Thus, it is better to exclude them for now, which will be mentioned in Section 14.

### 4.3 Label definitions

The set of labels reflects the judgement of the fact checker, and hence the four labels have different meaning: true means that the claim was found to be true by the fact checker, false means that the claim was found to be false, mixture means that the claim contained both true and false information, and unproven means that the fact-checker did not find enough evidence to prove or disprove the claim. Only the first two labels give a binary outcome, so only these were retained for the task.

This comes at a price since mixture and unproven claims constitute a significant part of the data and ideally a fact-checking system would handle all of them. While restricting the data makes the task more straightforward and better metrics possible, the models are never exposed to the kinds of claims that are often most challenging to deal with. The implication of this is discussed in Section 12

### 4.4 Record counts

Table 1 presents the number of records after applying the filter. Thus, in the training split, 9,832 records were imported. In the label filter, 8,079 claims labelled True and False are kept: 5,078 True and 3,001 False. From the training records excluded (1,753), 1,434 are mixture, 291 are unproven, and 28 are malformed. Moreover, in the test split, 987 of the 1,235 records were kept, including 599 True and 388 False. Finally, the validation split consists of 1,221 records in the tab-separated release. After the cleaning step, 1,214 records remained; however, they were not used because no tuning was carried out.

**Table 1: Records used in the project, by split**

| Split | Raw records | Used (True / False) | Excluded |
|---|---|---|---|
| Train | 9,832 | 8,079 (5,078 / 3,001) | 1,753 (1,434 mixture, 291 unproven, 28 malformed) |
| Validation | 1,221 | Loaded and cleaned (1,214), not used | Not used for tuning |
| Test | 1,235 | 987 (599 / 388) | 248 (mixture, unproven or blank) |

### 4.5 Class distribution

Figure 1 shows the label distribution in the training data. About 63% of the claims used for training are True (62.9%), so the classes are uneven. This matters in two places. It affects how a classifier learns, because a model trained without adjustment can achieve reasonable accuracy by favouring the majority class. It also affects how accuracy should be read. A model that always predicts True would score 60.7% on the test set, which is the baseline I report in Section 10.

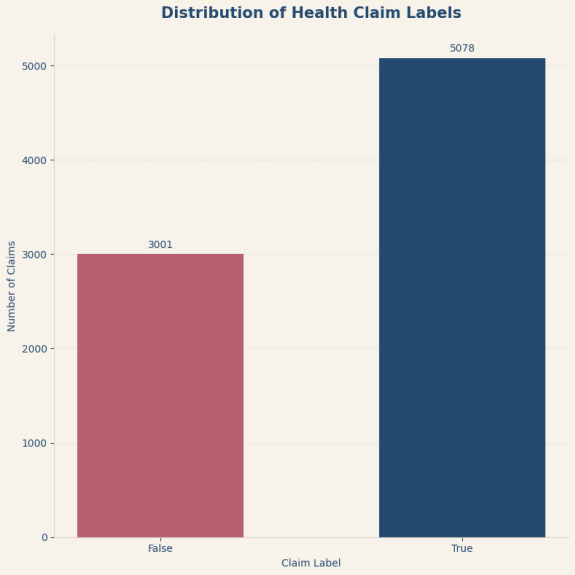



*Figure 1: Label distribution of the training claims used (True and False only).*

### 4.6 Data quality and cleaning

Inspection of the tab-separated files revealed a small number of malformed rows. In these rows, some of the text had spilled into the label column, so the label was empty or contained a long string rather than a verdict. Twenty-eight such rows appeared in the training split. They were removed by the label filter, because they could not be assigned a valid label. Blank claims were removed in the same step.

The dataset is written in English, and I did not find evidence of other languages in the claims I inspected, so no language filtering was applied. The licence is not stated on the dataset page. The data is used here only for academic coursework, and it is not redistributed in the repository. The repository instead records where to download it.

### 4.7 Known characteristics of the data

The claims cover many subjects, but the dataset contains a substantial amount of COVID-19 content, reflecting the period in which the claims were collected. This is important for interpretation. Some of the words with the strongest weights in the model, such as "coronavirus", reflect that topical emphasis. A model trained on different claims from a different period could give different weights. The dataset also reflects the editorial choices of the fact-checkers, who decide which claims to check in the first place. The model learns those choices as well as the verdicts.

### 4.8 Provenance and permitted use

Public datasets are not always free to reuse in every setting. The PUBHEALTH data is distributed for research, and the dataset page does not give an explicit licence. Using it for an academic project, as done here, is within the normal expectations of research use. Using it commercially, or redistributing the raw files, would need a clearer position from the original authors. For that reason, the repository links to the source and does not include the data itself. A reader who wants the data should obtain it from the original location and cite the original paper [1].

The fact-checkers' explanations and article text are also copyrighted material belonging to the fact-checking organisations and to the original publishers. They were not reproduced in this report, and the report quotes only the statistical properties of the dataset, not the content of individual records.

### 4.9 Risks in the data

Three risks in the data deserve attention. The first is label noise: fact-checkers are human, and their verdicts can be disputed, revised or influenced by the topic. The second is selection bias: a claim is only in the dataset if a fact-checker chose to check it, so the claims are not a random sample of what people believe or share. The third is temporal drift: the dataset reflects the period in which the claims were checked, and the language of misinformation changes quickly. Each risk limits how far the results can be generalised, and each is relevant to the subgroup findings in Section 10.

Of the three, selection bias is the hardest to address with this dataset. A model trained on checked claims learns to recognise what fact-checkers choose to check, which may differ from what is harmful. A claim that is widely believed but never checked is simply absent from the training data, so the model cannot learn from it. This is a fundamental limit of supervised learning on fact-checked data, and it applies to any system built this way.

## 5. Data Preprocessing

Preprocessing in this project was kept deliberately minimal. The reason is interpretability. Each feature in the final model is a word or two-word phrase that appears in the claim. If I had removed stop words, stemmed the text or normalised spelling, the features would still be readable, but the explanation would describe a transformed version of the text that a reader would find harder to recognise. I therefore decided to change the text as little as possible and to report the consequences of that choice honestly.

### 5.1 Steps that were performed

- **Removing empty and malformed rows.** Rows without a claim or without a valid label were dropped before any other step. Leading and trailing whitespace was stripped from each claim.
- **Label filtering and encoding.** Only claims labelled True or False were kept. True was encoded as 1 and False as 0, so that the classifier's positive class is True.
- **Lowercasing.** The TF-IDF vectoriser lowercases all text by default. "Vaccines", "vaccines" and "VACCINES" therefore map to the same feature.
- **Tokenisation.** The vectoriser splits text into tokens using the pattern \b\w\w+\b. This means a token is a run of at least two word characters. Punctuation is discarded, and single-character tokens such as "a" and "I" are ignored.
- **Phrase construction.** Unigrams and bigrams are both produced, so two-word phrases such as "in children" or "face mask" become features in their own right.

### 5.2 Steps that were deliberately not performed

Stop-word removal was not used. Stop words are very common words such as "the", "is" and "and". Many NLP pipelines remove them, on the grounds that they carry little topical information. I kept them for two reasons. First, the explanation analysis needed to test whether the model relies on them, and that test is impossible if they are removed in advance. Second, the main question of the project is

about bias in how claims are written, and stop words are one of the main carriers of writing style. Removing them would have hidden the very effect I wanted to measure. The consequence is that the model learns strong weights on words such as "the" (weight −5.52, the largest magnitude in the model), which is discussed in Section 10.

No stemming or lemmatisation was applied. Stemming would merge forms such as "cure", "cures" and "cured", which could help a larger model, but it would also make the explanations less precise. No emoji handling, URL removal, hashtag handling or mention handling was used. These steps are relevant to social media text, but the PUBHEALTH claims are fact-checked statements from news and fact-checking sites rather than posts. Finally, no spelling correction was performed, because the claims were written by the sources and should be taken as they were published.

### 5.3 Why the choices matter

Every preprocessing choice shapes what the model can learn. Keeping stop words means the model can use grammatical structure as a signal, which may help with some claims and may hurt others. Keeping case folding and punctuation removal means the model is not sensitive to capitalisation or to the difference between "COVID-19" and "covid 19", which is helpful, but it also means that numerical claims are represented poorly. The token pattern ignores single-digit numbers, so a claim about "a 1 percent risk" loses the number entirely. These are small effects, but they are the kind that a reader should know about when they look at an explanation.

### 5.4 Alternatives considered

I considered three alternatives before settling on minimal preprocessing. The first was a standard English stop-word list, which would have reduced the vocabulary and possibly the length bias. The second was lemmatisation, which would have merged word forms but required an additional dependency and a more complicated explanation. The third was to keep the default settings but add a length feature, so that the model could learn length directly. I did not adopt the first two because they would change the very features the project is supposed to explain. I did not adopt the third because it would have treated length as a feature to be learned rather than as a condition to be audited.

### 5.5 How preprocessing shapes the explanations

Preprocessing does not only change the accuracy of a model. It also changes what the explanation can say. If the vectoriser lowercases the text, the explanation cannot distinguish "COVID" from "covid", which is harmless here but would matter for a name or an abbreviation with a different meaning in capitals. If the tokeniser drops single-character tokens, the explanation cannot mention the letter "a" or a single digit, even when that character is part of a meaningful phrase. These are small effects, but they limit what a reader can learn from the output.

The choices also interact with the length audit. Keeping stop words means that longer claims have more features that the model can weigh against each other, and this is part of the reason the length effect appears. Removing stop words would reduce the number of features in long claims, and the audit could then show whether the length effect is caused by function words or by the content of the claims. That is one of the most useful experiments left open by this project.

## 6. Feature Representation / Embeddings

Every claim is represented as a vector of TF-IDF weights [4], in the form described in standard information-retrieval texts [14]. TF-IDF stands for term frequency–inverse document frequency. For a term t in a claim d, the term frequency is how often t appears in d. The inverse document frequency is a measure of how rare t is across the whole training collection, usually computed as the logarithm of the total number of documents divided by the number of documents that contain t. Multiplying the two gives a weight that is high for terms that are frequent in a particular claim and rare across claims, and low for terms that appear almost everywhere.

In scikit-learn, the TfidfVectorizer uses a smoothed version of the inverse document frequency and normalises each vector to unit length with the l2 norm. The vectoriser in this project uses the default settings for these details, with use_idf set to True and smooth_idf set to True. The vector for each claim therefore has length one, which prevents long claims from receiving larger feature values simply because they contain more words.

### 6.1 Parameter choices

- **ngram_range = (1, 2).** Both unigrams and bigrams are included. Unigrams capture individual words, while bigrams capture short phrases that often matter more than their parts, such as "not effective" or "in children".
- **min_df = 2.** Any term that appears in fewer than two training claims is dropped. This removes one-off words that would otherwise let the model memorise individual training claims.
- **max_features = None.** No cap is placed on the vocabulary size. The vocabulary that results from the filters is the full set of terms the model can use.
- **lowercase = True.** All text is lowercased before tokenisation, as discussed in Section 5.

### 6.2 Resulting feature space

After the training data is processed, the vocabulary contains 17,035 features. Of these, 6,988 are single words and 10,047 are two-word phrases. The number of bigrams is larger than the number of unigrams, which is typical for short texts: most pairs of adjacent words are rarer than the individual words that make them up, but a large number of distinct pairs still survives the min_df filter. Each claim activates only a small fraction of these features, so the representation is sparse. This sparsity is one reason the model trains quickly on a laptop.

Because every feature corresponds to a specific string of words, the representation can be inspected directly. A feature such as "coronavirus" has a clear meaning, and so does "the common cold". This is the property that the explanation method depends on, and it is the main reason TF-IDF was chosen over alternatives.

### 6.3 Why not contextual embeddings

Contextual embeddings from models such as BERT [10] usually give higher accuracy on text classification. They map a sentence to a dense vector whose dimensions do not correspond to words, so an explanation has to come from a separate method. Methods such as LIME [11] can approximate a prediction locally, but the approximation is itself a model and has its own uncertainty. For this

project, where the purpose is to explain and audit a classifier, I preferred a representation whose explanation is exact rather than approximate.

The second reason is the size of the data. A few thousand labelled claims are enough to train a sparse linear model reliably, but they are small by the standards of pretraining. Fine-tuning a large model on so little data would require careful regularisation and a more elaborate evaluation, which would have moved the project away from its aims. The third reason is practical. The website runs on a small Flask server, and a linear model with 17,035 weights is easy to host, whereas a transformer would need more memory and a more demanding deployment.

### 6.4 Limitations of the representation

TF-IDF ignores word order beyond the two-word window, so it cannot represent negation reliably. The phrase "is not effective" contributes features for "is", "not" and "effective", together with the bigrams "is not" and "not effective", but the model has no concept of negation as such. It also cannot represent meaning across synonyms: "cure" and "treat" are separate features. These limits are part of why the model is weakest on longer and more complex claims, a point developed in Section 11.

### 6.5 A worked example of the TF-IDF calculation

To make the weighting concrete, consider a toy collection of three claims: "Vaccines cause autism in children.", "Vaccines are safe for children." and "Drinking water cures every disease." With n = 3 documents, scikit-learn's smoothed inverse document frequency for a term that appears in df documents is idf = ln((1 + n) / (1 + df)) + 1. A term that appears in only one claim therefore receives idf = ln(4/2) + 1 = 1.693, while a term that appears in two claims receives idf = ln(4/3) + 1 = 1.288. Terms such as "vaccines" and "children", which appear in two claims, receive the lower weight, and terms such as "autism" and "cause", which appear only once, receive the higher weight.

The raw TF-IDF value is the term frequency multiplied by the idf. For each claim, the resulting vector is then normalised to unit length using the l2 norm. In the first claim, the vector has entries of about 0.37 for "vaccines" and "children" and about 0.49 for "autism", after normalisation. These are the values that the logistic regression multiplies by its weights. The normalisation step means that the total magnitude of a claim's features is fixed, so a long claim does not automatically receive larger feature values than a short one.

The example shows why rare words get more weight than common ones, and why this helps a classifier. It also shows a side effect: a word that appears in every claim of a collection receives the lowest possible idf, which means that TF-IDF already downweights many function words. This is a partial solution to the stop-word problem, and it is the reason the model's largest function-word weights are interesting. The function words in the real dataset are common, but they are not common enough to be downweighted to zero, and their learned coefficients still reflect the labels.

### 6.6 Why only unigrams and bigrams

Longer n-grams were not tested. A trigram such as "in children with autism" would be rarer still, so most trigrams would fall below the min_df threshold and be dropped. The vocabulary would grow more slowly, but the features would be more specific and less likely to generalise. Bigrams were the compromise: they capture short phrases of negation, qualification and emphasis, such as "not

effective" and "safe to", while keeping the feature space manageable. Whether trigrams would help is an open question that the future work in Section 14 addresses.

## 7. Proposed Methodology / System Architecture

### 7.1 Overview of the pipeline

The system follows the workflow shown in Figure 2. A claim from the PUBHEALTH splits is cleaned and filtered. It is converted into TF-IDF features by the vectoriser, and the resulting vector is passed to the Logistic Regression classifier, which produces a probability for each class. The label is the class with the higher probability. The same fitted pipeline is then reused in three places: the length audit, which measures performance on subgroups; the explanation step, which reads the coefficients; and the website, which exposes the model to users.

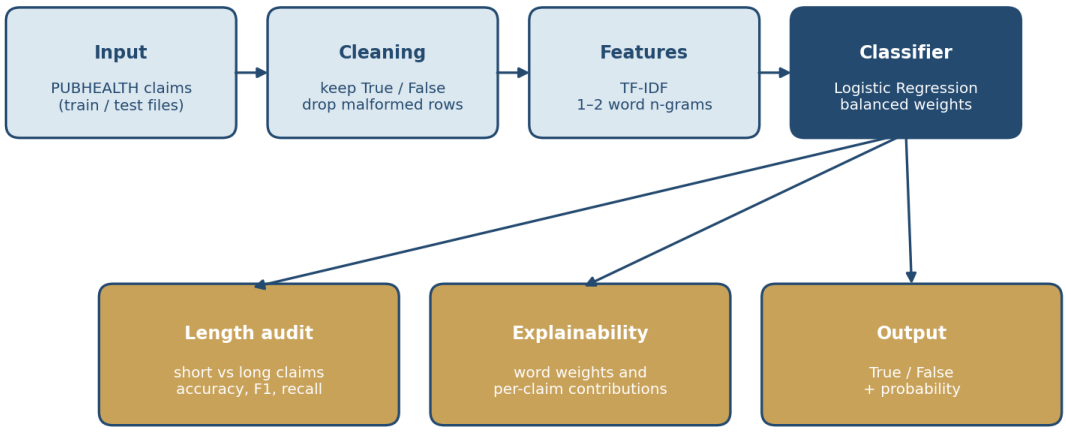



*Figure 2: Workflow from the input claim to the outputs of the audit, the explanation and the website.*

### 7.2 Experimental protocol

The model is trained only on the training split, which contains 8,079 claims. All reported results come from the test split, which contains 987 claims. The test split was not used to choose any setting. Because I made no hyperparameter search, there is no selection step that could have leaked information from the test set into the results.

The validation split was loaded and cleaned, but it was not used. This is a limitation, because validation data is normally used to tune settings such as the regularisation strength and the decision threshold. The default settings were kept, so the validation split was not needed, and the test results are reported as the only evaluation.

### 7.3 Subgroup definition

The main subgroup analysis compares claims of 15 words or fewer with claims of more than 15 words. I chose 15 words before running the audit, using a rough sense of what a short claim looks like rather than any result from the model. The number of words is counted by splitting the claim on whitespace. This definition is simple and transparent, but it is not the only possible one. A claim can

be long because it is complicated or because it contains many function words, and the length measure does not separate those causes. I return to this in Section 11.

### 7.4 Metrics

Four metrics are reported. Accuracy is the proportion of claims whose predicted label matches the true label. Precision for the True class is the proportion of claims predicted True that are truly True. Recall for the True class is the proportion of truly True claims that the model labels True. The F1 score for a class is the harmonic mean of precision and recall, and macro-F1 is the unweighted average of the F1 scores for the two classes.

I report precision and recall for the True class because it is the positive class in the notebook. The macro-F1 gives the two classes equal weight, so it is less flattering to a model that favours the majority class. Accuracy is still reported because it is the number most readers expect, but I do not rely on it alone. The reasons for this are discussed in Section 10.

### 7.5 Explanation method

The explanation method uses the structure of the linear model. For a claim x, the decision score is the sum over features of x_j multiplied by the coefficient w_j, plus an intercept. Each term in that sum is the contribution of feature j to the decision. Contributions above zero push toward True, and contributions below zero push toward False. Because the sum is exact, the explanation shows the real reasons for the model's output, not a simulation of them.

This method has a limitation. The contributions depend on the TF-IDF weights, which are normalised across the whole claim. Adding a long sentence therefore changes the weights of every term in the claim, not only the weights of the new terms. The explanation is faithful to the model, but a reader should remember that it describes the model's arithmetic rather than the claim's meaning.

### 7.6 Reproducibility

The random state is not relevant to this pipeline, because Logistic Regression with the lbfgs solver is deterministic on a given dataset. The pipeline was fitted once and saved to a single file, healthguard_model.joblib, which the server loads. Anyone with the notebook and the PUBHEALTH files should obtain the same coefficients, provided they use the same scikit-learn version. The saved model was produced with scikit-learn 1.6.1, and the repository's requirements file pins that version.

### 7.7 Threats to validity

Every empirical study has threats to its conclusions. Four categories are relevant here, following the usual distinction between internal, external, construct and statistical validity.

- **Internal validity.** The results depend on decisions I made during development, such as the 15-word threshold and the choice to keep stop words. These decisions were made before the audit, and the threshold was not tuned on the test set, but a different threshold would give a different table. The findings should be read as the result of these particular choices.
- **External validity.** The results apply to English claims from the PUBHEALTH fact-checking sources, and to the period in which those claims were checked. They do not show how the model would perform on social media, on claims in other languages, or on claims published after the dataset was assembled.

- **Construct validity.** The labels are a proxy for truth. The model is evaluated against fact-checker verdicts, so its accuracy measures agreement with those verdicts, not agreement with the facts. The word count used to define length is also a proxy: it does not measure complexity, hedging or the number of distinct claims in a sentence.
- **Statistical conclusion validity.** The test set has 987 claims, and the long-claim subgroup has 270. The differences reported in Section 10 are large, but the estimates for the long group are not precise, and only one split was used. Repeated sampling would be needed to put tight intervals around them.

### 7.8 Record of design decisions

Table 2 summarises the main decisions, the alternatives considered and the reasons for the choice. It is included so that a reader can see which parts of the design were deliberate and which were defaults.

**Table 2: Main design decisions**

| Decision | Alternative considered | Reason for the choice |
|---|---|---|
| Keep stop words | Remove a standard stop-word list | The bias analysis needs them; removing them would hide the effect |
| Unigrams and bigrams | Unigrams only; trigrams | Captures negation and short phrases; keeps the vocabulary manageable |
| min_df = 2 | No frequency filter | Removes one-off terms that would let the model memorise claims |
| Logistic Regression | Naive Bayes, SVM, Random Forest, SGD | Readable weights and direct probabilities; similar performance on this data |
| Balanced class weights | Unweighted training | About 63% of the training claims are True |
| Default C = 1.0 | Grid search over C | Avoids a second round of choices the audit would have to explain |
| 15-word subgroup | Character count, sentence count, or no split | Simple, objective and fixed before the audit |
| Exact per-claim explanation | LIME or SHAP approximation | The contributions are exact for a linear model |
| Website served by Flask | Static HTML with the model in JavaScript | Keeps one model file and one source of truth |

## 8. NLP Model / Technique Used

### 8.1 The classifier

The model is a two-step scikit-learn pipeline [3, 7]. The first step is the TfidfVectorizer described in Section 6. The second step is a LogisticRegression classifier with the lbfgs solver, a maximum of 1,000 iterations, the default regularisation strength C = 1.0, and balanced class weights. The pipeline is fitted once on the training claims. The same object is then used for prediction, evaluation, explanation and the website, so that every output comes from the same model.

### 8.2 How Logistic Regression works here

Logistic Regression models the probability that a claim is True as the logistic function of a linear combination of its features. The linear combination is the sum of each feature value multiplied by a

learned weight, plus an intercept. The logistic function squashes this sum into a value between zero and one. If the probability is above 0.5, the model predicts True.

The learned weights carry the meaning of the model. A large positive weight means that the presence of the corresponding feature raises the probability of True. A large negative weight means it lowers the probability. The intercept is the baseline: it is the score the model would give a claim with no recognised features. In this model the intercept is 0.712, which is positive, so the baseline leans toward True. This is a consequence of the class balance in the training data, and it has a practical effect discussed in Section 11. A claim whose words the model does not recognise is labelled True with about 67% probability.

### 8.3 Class weights

About 63% of the training claims are True. A classifier trained without adjustment is pushed toward the majority class, because mistakes on that class reduce the training loss more often. The class_weight = "balanced" setting addresses this. Each class is weighted inversely to its frequency, so that the total weight of the True claims and the total weight of the False claims are equal during training. In effect, an error on a False claim counts as much as an error on a True claim, even though there are fewer False claims.

Class weights change the decision boundary. They do not change the features, so the explanation of individual predictions is still based on the same weights. However, the weights can change how the intercept and the other coefficients are estimated, so they are part of what the explanation reflects.

### 8.4 Settings that were not tuned

I did not search over the regularisation strength, the decision threshold, the n-gram range or the minimum document frequency. Each of these could change the results, and some of them probably would. The decision was deliberate. The project is about auditing a baseline, and tuning would have introduced a second round of choices that the audit would then have to explain. Tuning is listed as future work in Section 14.

### 8.5 Comparison with other classifiers

To check whether the choice of classifier mattered, I also trained Naive Bayes, Linear SVM, Random Forest and Stochastic Gradient Descent on the same features. The alternatives performed in a similar range to Logistic Regression, as shown in Figure 7. Naive Bayes and Linear SVM are both suitable for sparse text, and the fact that they perform similarly suggests that the features, rather than the learning algorithm, limit the results. A Random Forest can capture interactions between features, but on this data it did not add a clear benefit.

I kept Logistic Regression for the final pipeline for two reasons. Its coefficients can be read as a single weight per feature, which the explanation method depends on. And its probabilities are reported directly, which the website displays. The other models would have required a different and less direct explanation method.

### 8.6 Model size

The fitted model stores one coefficient for each of the 17,035 features, together with the intercept and the vectoriser vocabulary. Every coefficient is non-zero, because Logistic Regression with L2

regularisation rarely sets weights exactly to zero. The saved file is about 520 kilobytes. This is small enough to load quickly when the server starts, and it means the model could be used on a modest machine without a graphics card.

### 8.7 Interpreting the intercept

The intercept of 0.712 deserves a closer look, because it determines what the model does with text it does not recognise. For a claim that contains no features from the vocabulary, every feature value is zero, so the decision score equals the intercept. The logistic function of 0.712 is about 0.67, which means the model would label such a claim True with 67% probability. The probe "xyzzy quux blorp" gives exactly this result.

The positive intercept is a product of the training data. Because about 63% of the training claims are True, and the classes are balanced by weighting, the intercept is set to reflect the base rate in the weighted data. In practice this means the model has a built-in lean toward True whenever the evidence is weak. For a health classifier this is the wrong default. A safer system would either refuse to classify unrecognised input or would default to an explicit "cannot determine" output. This is one of the clearest recommendations of the project.

## 9. Implementation

### 9.1 Tools and environment

The notebook was written in Python 3 and run in Google Colab [5]. The main libraries are pandas and NumPy for data handling, scikit-learn [3] for the vectoriser, the classifier and the metrics, and matplotlib for the charts. The web application uses Flask [6] as a small server, joblib to load the saved model, and plain HTML, CSS and JavaScript for the page. No frontend framework was used, so the page can be opened and edited with any text editor.

Development was done in two places. The notebook was developed in Colab so that the data could be loaded from the PUBHEALTH files without local setup. The website was developed on a Windows laptop, with the model file copied into the project folder. This split meant the notebook and the website had to agree on one file format, which is why the model is saved with joblib and loaded by the server rather than retrained.

### 9.2 Modules

The implementation is divided into four parts. Each has a single responsibility, which made the project easier to test and to explain.

- **Data loading and filtering.** Reads the PUBHEALTH train, validation and test files, removes malformed rows, keeps the True and False claims, and encodes the target. This code appears at the start of the notebook.
- **Training and evaluation.** Fits the pipeline on the training claims, predicts on the test claims, and reports the overall metrics, the length audit and the confusion matrix.
- **Explainability.** Reads the coefficients of the fitted classifier, lists the strongest words and phrases for each class, and produces the weight charts used in Section 10.

- **Model export and serving.** The notebook saves the fitted pipeline to healthguard_model.joblib. The server file app.py loads that file once at start-up and answers requests on the /predict route.

### 9.3 The prediction endpoint

The server exposes one prediction route. A client sends a JSON object containing the claim, for example {"claim": "Vaccines cause autism in children."}. The server checks that the claim is not empty and that it is no longer than 3,000 characters. If the check passes, it returns a JSON object with three fields: the prediction, which is 1 for True and 0 for False; the confidence, which is the probability of the predicted class rounded to three decimal places; and the explanation, which lists the number of matching terms and the largest contributions toward each class.

The server returns an error response with status 400 when the claim is missing or too long, so that the page can show a clear message instead of failing silently. The server does not write claims to disk and does not keep a log of them, so the claims entered by users are not stored.

### 9.4 How the explanation is computed

For a claim, the server transforms the text with the fitted vectoriser to obtain the sparse TF-IDF vector. It then multiplies each non-zero entry by the coefficient of the corresponding feature. The product is that feature's contribution to the decision score. The server sorts the contributions into two lists: the positive contributions, which push toward True, and the negative contributions, which push toward False. It returns the five largest from each list, together with the number of matching features.

Two details matter for interpretation. First, the contributions are computed after normalisation, so they describe the actual inputs to the model. Second, the intercept is not included in the list. The intercept is the baseline, and it applies to every claim, so it does not explain why one claim differs from another. When a claim contains no recognised terms, the explanation says so, and the reader is reminded that the answer comes from the baseline.

### 9.5 The website

The website is a single HTML page, with its styling in the same file and its background image in a separate file. A user pastes a claim into a text box and clicks the Analyse claim button. The page checks that the box is not empty, sends the claim to /predict, and displays the result. The result shows the predicted label, the reported model confidence, and a panel titled Why this result. The panel lists the terms that pushed the answer toward True and toward False, with their weights. Below it, a short paragraph explains that the model compares the claim with patterns in the PUBHEALTH training data, and that it does not check the claim against a source.

The page also carries a standing notice. It states that the output is an automated classification, that it is not proof that a claim is true or false, and that health decisions should be made with a qualified professional. These notices are visible on every result, so that a reader does not need to scroll past them to find them.

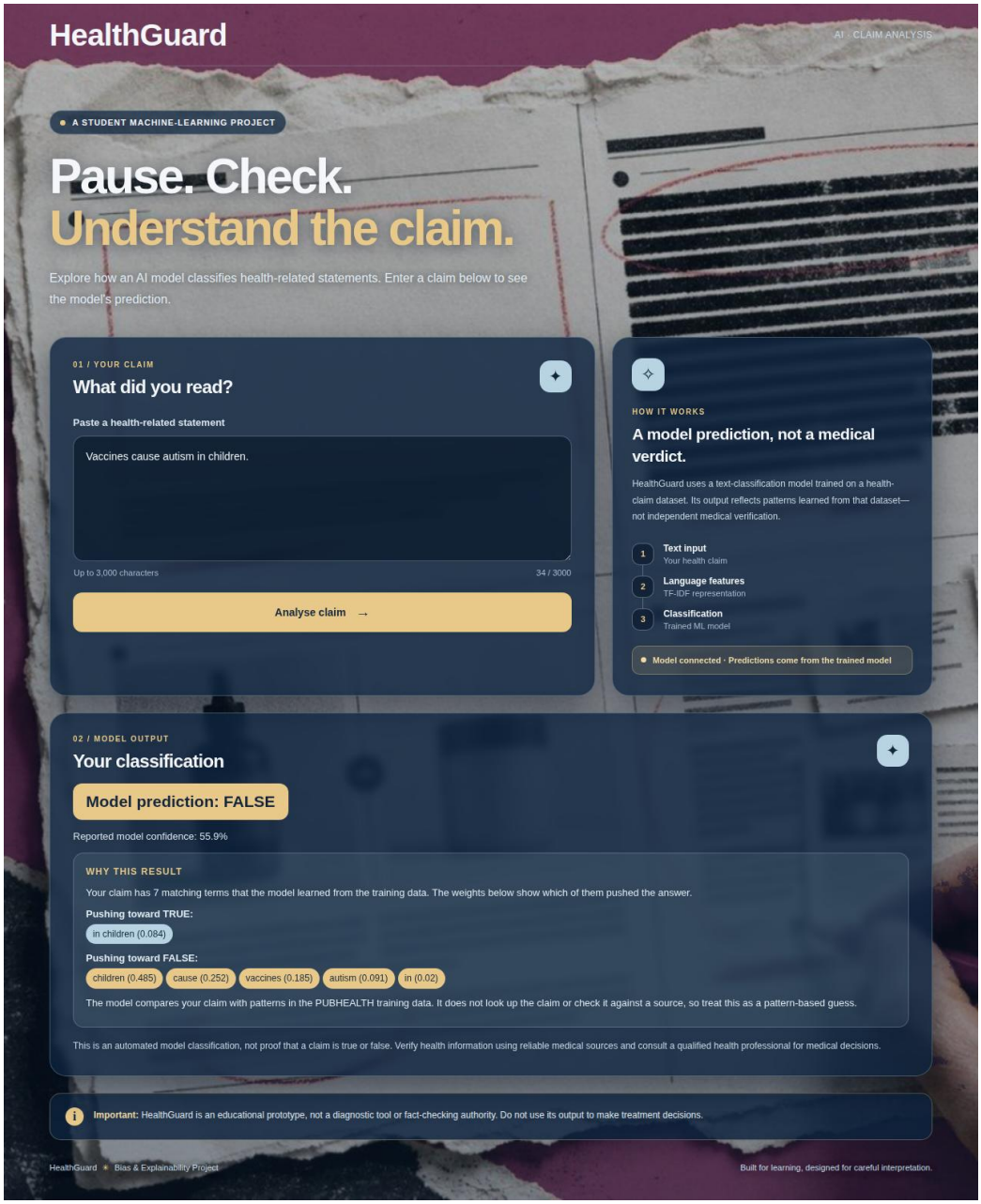



*Figure 3: The website after a prediction, showing the label, the confidence and the explanation for "Vaccines cause autism in children."*

### 9.6 Testing

I tested the system at three levels. At the model level, I checked that the saved file loads under the pinned scikit-learn version without warnings, and that the same claim returns the same probability on repeated calls. At the server level, I used the Flask test client to confirm that the page loads, that /predict returns a 200 response with the expected fields for valid input, and that it returns a 400 response for an empty claim. At the page level, I ran the site in a headless Chromium browser, entered claims, and checked that the label, the confidence and the explanation appeared.

The tests were informal, and they are not a substitute for a proper test suite. They did reveal a few real issues during development, including a repeated number in the explanation sentence, which was

corrected, and a background image that did not span the full page when the browser captured the whole page. Both were fixed before the final version.

### 9.7 Running the system

To run the system, the user installs the packages listed in requirements.txt and starts app.py. The server then listens on port 5000, and the page is available at localhost:5000. A launcher file, Start_HealthGuard.bat, performs both steps on Windows, so that a user who does not want to use the command line can start the site with a double click. The launcher installs the requirements the first time it runs, starts the server and opens the page in the default browser.

### 9.8 Specification of the prediction interface

Table 3 summarises the interface between the website and the server. It is written so that another developer could build a different front end, or a different client, against the same server without reading the code.

**Table 3: Prediction interface**

| Item | Specification |
|---|---|
| Route | /predict (POST); / (GET) returns the page |
| Request body | JSON object with one field, claim (string) |
| Validation | claim must not be empty after trimming, and must not exceed 3,000 characters |
| Success response | JSON with prediction (0 or 1), confidence (float, rounded to three decimals) and explanation |
| Explanation object | known_terms (integer), toward_true and toward_false (lists of word and weight pairs, at most five each) |
| Error response | HTTP 400 with an error message for a missing or over-long claim |
| Storage | None. Claims are not written to disk or logged |

### 9.9 A worked example of one explanation

The explanation is easiest to trust when its arithmetic can be checked by hand. Take the claim "Vaccines cause autism in children." The server finds six features in the vocabulary, with the following contributions to the decision score: "children" −0.485, "cause" −0.252, "vaccines" −0.185, "autism" −0.091, "in" −0.020 and "in children" +0.084. Adding these gives a total of −0.949. The intercept of +0.712 is then added, giving a decision score of −0.237.

The logistic function of −0.237 is 1 / (1 + e^0.237), which is about 0.441. That is the probability of True, so the probability of False is about 0.559. The model predicts the more likely class, which is False, and reports 55.9% as the confidence. The website displays exactly these numbers, and the explanation lists the largest contributions on each side. The arithmetic can be repeated by anyone with the saved coefficients.

The example also shows the limits of the explanation. The six contributions sum to −0.949 and the intercept adds +0.712, so the intercept accounts for a large share of the final score. A reader who looked only at the listed words would see a strong negative pull from "children", but would not see that the baseline partly offsets it. This is why the explanation panel reports the number of matching terms and says that the answer depends on the baseline when no terms match.

### 9.10 Deployment and hosting options

The project runs on a local machine, which is enough for demonstration and for a classroom review. Making it available to other people would need a hosting environment that can run a Python server. Several services offer this, and the choice depends on cost, on the expected traffic and on whether the model file can be stored with the code. The model is small, about half a megabyte, so storage is not a constraint. The server is also light, because it loads the model once and answers each request quickly.

Before any public deployment, three things would need to change. The page would need a clear privacy notice that explains what is and is not stored. The server would need rate limiting, so that the endpoint cannot be overwhelmed. And the output would need a stronger warning, or a different presentation, so that a user does not treat the label as a verdict. None of these changes is difficult technically, but each requires a decision about how the system should be presented, which is a judgement that belongs with the people responsible for its use.

Monitoring is the final requirement. A deployed model should log aggregate statistics such as the share of predictions labelled True and the distribution of confidence values, without storing the claims themselves. Shifts in those statistics would indicate that the input has changed, and that the audit should be repeated on new data. This is a standard practice for deployed models, and it is absent from the prototype.

## 10. Results and Evaluation

### 10.1 Overall performance

On the test set of 987 claims, the model labels 753 claims correctly, which is 76.3% accuracy. The majority-class baseline, which predicts True for every claim, labels 599 claims correctly, or 60.7%. The model therefore improves on the baseline by 15.6 percentage points. That is a real improvement, but it is smaller than the headline accuracy might suggest, because a large part of the gain comes from the class balance rather than from any understanding of the claims.

Precision and recall are reported for the True class. The precision is 0.837, meaning that of the claims the model labels True, about 84% are truly True. The recall is 0.756, meaning that the model finds about 76% of the truly True claims. For the False class, the precision is 0.673 and the recall is 0.773. The macro-F1, which averages the two class F1 scores equally, is 0.757. Table 4 gathers these numbers.

**Table 4: Overall results on the test set (n = 987)**

| Model | Accuracy | Precision (True) | Recall (True) | Macro-F1 |
|---|---|---|---|---|
| Majority baseline (always True) | 60.7% | — | 100% | 0.378 |
| HealthGuard (TF-IDF + Logistic Regression) | 76.3% | 0.837 | 0.756 | 0.757 |

### 10.2 Confusion matrix

Figure 4 gives the full breakdown. Of the 388 False claims, 300 are labelled False and 88 are labelled True. Of the 599 True claims, 453 are labelled True and 146 are labelled False. The errors are therefore unequal. The model more often misses a True claim (146 cases) than it accepts a False claim as True (88 cases). In a fact-checking setting, the first kind of error means a true claim is

flagged or dismissed incorrectly, while the second means a false claim is missed. Which error matters more depends on how the output is used, and the report returns to this in Section 12.

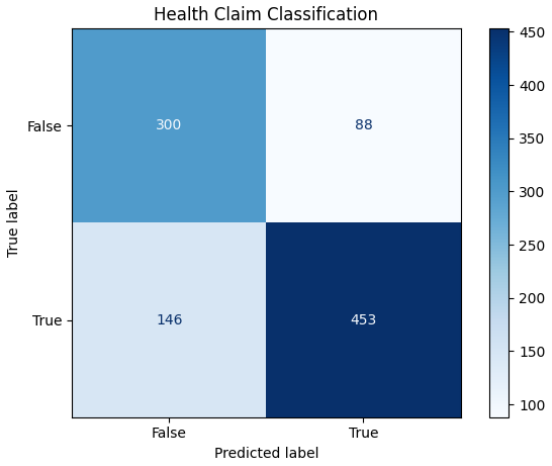



*Figure 4: Confusion matrix on the test set. Rows are the true labels; columns are the predictions.*

### 10.3 Length audit

The audit is the central result of the project. Table 5 splits the test set into short and long claims and reports the same metrics for each. Short claims are those of 15 words or fewer, and there are 717 of them. Long claims are those with more than 15 words, and there are 270.

**Table 5: Results by claim length on the test set**

| Group | Definition | n | Accuracy | Precision (True) | Recall (True) | Macro-F1 |
|---|---|---|---|---|---|---|
| Short | 15 words or fewer | 717 | 79.8% | 0.872 | 0.844 | 0.754 |
| Long | More than 15 words | 270 | 67.0% | 0.400 | 0.198 | 0.526 |

Accuracy falls by 12.8 percentage points on long claims. The recall figures show the larger problem. Only about one in five long True claims (19.8%) is labelled True, compared with about four in five short True claims (84.4%). The precision for long True claims is 0.400, which means that when the model does label a long claim True, it is often wrong. In other words, the model is unreliable for long claims in both directions, and the errors are concentrated in the True class.

Macro-F1 falls from 0.754 for short claims to 0.526 for long claims. Because macro-F1 gives the two classes equal weight, this drop shows that the problem is not only the larger False class. The model is substantially weaker on the True claims within the long group, and accuracy alone hides this, because the long group contains many False claims that the model handles reasonably well.

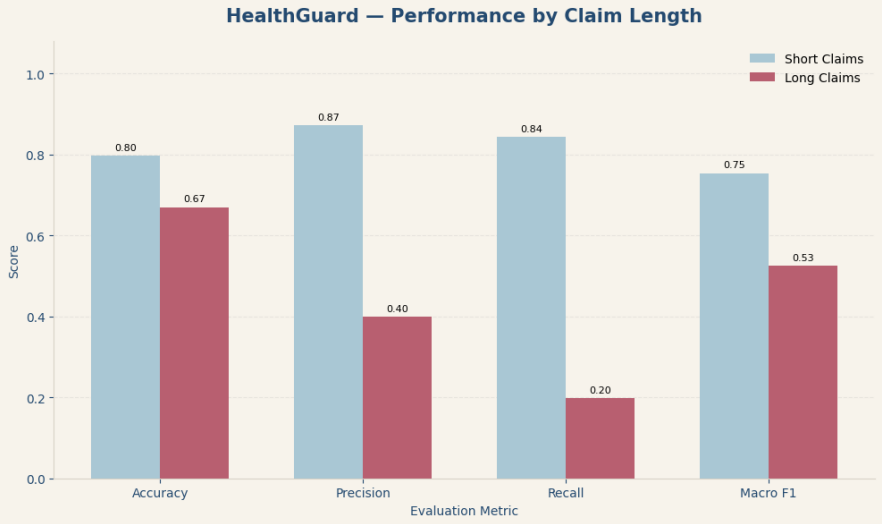



*Figure 5: Performance by claim length on the test set, short claims (15 words or fewer) and long claims (more than 15 words).*

The long-claim group has 270 test claims. That is enough to see a clear difference, but it is not enough to estimate the true-positive rate precisely. A confidence interval around 19.8% from 270 examples is wide, and the result should be confirmed with more long claims before anyone relies on the exact figure. The direction of the difference is much clearer than its size.

### 10.4 Word-level explanation

Figure 6 shows the strongest word weights in the model. The positive side is led by "coronavirus" (weight 2.45), "some" (1.997), "report" (1.812), "vaping" (1.768), "us" (1.708), "mental health" (1.685), "study" (1.683), "health" (1.640), "officials" (1.619) and "uk" (1.599). The negative side is led by "the" (−5.519), "is" (−3.741), "was" (−3.434), "and" (−3.425), "that" (−3.211), "an" (−3.156), "are" (−3.138), "or" (−2.646), "has" (−2.549) and "obama" (−2.455).

The two lists reveal two different kinds of signal. The positive side mostly contains topical or source-related words: some of them, such as "report", "officials" and "study", are typical of how a fact-checker describes evidence, and others, such as "coronavirus", "vaping" and "mental health", reflect the subjects that were common in the source data. The negative side consists almost entirely of function words and one proper name. Function words carry little information about whether a claim is true, so their large weights are a sign that the model is picking up on style.

The weight on "the" is the most striking. Its magnitude is more than twice that of the largest positive weight. Because "the" is very common, it appears in most claims, and the model learns that claims containing it tend to be labelled False. Whether this is a property of false claims in the data or simply a property of the way fact-checkers write is not something the model can tell us. Either way, it is a bias that a reader would not expect from a health classifier.

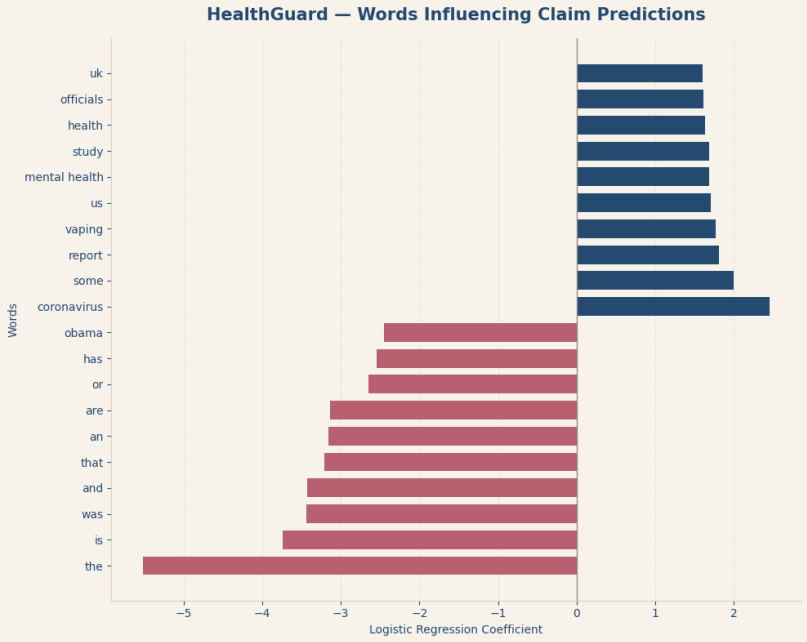



*Figure 6: Strongest word weights. Navy bars push toward True; berry bars push toward False.*

### 10.5 Comparison with other classifiers

Figure 7 compares Logistic Regression with the other classifiers trained on the same features. On this data, the alternatives give scores in a similar range, and none of them clearly outperforms Logistic Regression. The result suggests that the representation limits performance more than the choice of learner does. A more expressive classifier cannot recover information that the TF-IDF features do not contain, and the features here cannot represent negation, word order or meaning beyond the two-word window.

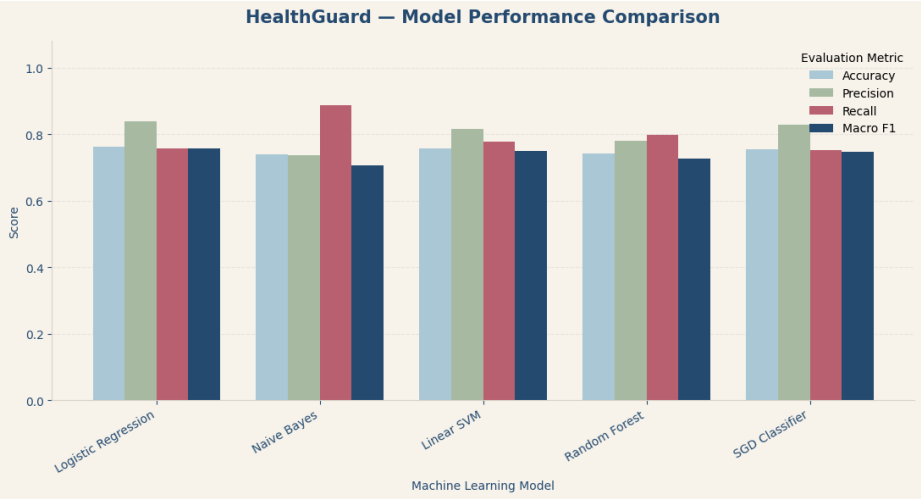



*Figure 7: Comparison of classifiers on the same TF-IDF features, scored on the test set.*

### 10.6 Answers to the research questions

RQ1 asked how accurately the model classifies claims. The answer is 76.3% accuracy and a macro-F1 of 0.757, clearly above the majority baseline. RQ2 asked whether performance differs by claim length. It does, substantially: accuracy drops by 12.8 points, and recall on long True claims is only

19.8%. RQ3 asked which words drive the decisions. The answer is that topical words push toward True, while common function words carry the largest weights toward False. The model therefore relies partly on how a claim is written, which means the explanations are useful for auditing but do not show that the model understands the claims.

### 10.7 What the results do not show

It is worth being clear about what the results cannot support. They do not show that the model is accurate for health claims in general, because the test set is a single sample from one dataset and one period. They do not show that the model understands medicine, because it has no representation of medical facts beyond the words that co-occur with labels. They do not show that the explanations are correct about the world, only that they are faithful to the model. And they do not show that the 15-word split is the best way to divide claims, because it was chosen as a simple and objective rule, not as the best one.

The results also do not compare the model with a human fact-checker, or with a stronger automated system. A reader who wanted to say that the model is "good enough" would need that comparison, along with a clear statement of the cost of each kind of error. This project provides the audit that such a comparison would need, but not the comparison itself.

## 11. Error Analysis / NLP Failure Analysis

The aggregate numbers show how often the model is wrong. The error analysis asks why. I focus on three kinds of failure: near ties, where the model is almost indifferent; style-driven errors, where the wording decides the label; and length-driven errors, where long claims are mis-scored for reasons that have little to do with their truth. The examples come from two sources. Table 6 uses claims I typed into the website as probes. They were not part of the test set, and I did not check them against any fact-checker. The point of the probes is to show the model's behaviour, not to measure its accuracy.

**Table 6: Probe claims, the model's label, and the terms that drove each decision**

| Claim | Label (P(True)) | Largest terms toward False | Largest terms toward True |
|---|---|---|---|
| "Vaccines cause autism in children." | FALSE (0.44) | "children" (−0.49), "cause" (−0.25), "vaccines" (−0.19) | "in children" (+0.08) |
| "Drinking water cures every disease." | TRUE (0.52) | "drinking" (−0.34), "every" (−0.32), "disease" (−0.06) | "drinking water" (+0.05), "cures" (+0.01), "water" (+0.01) |
| "Researchers are studying the relationship between sleep and health." | FALSE (0.31) | "the" (−0.73), "are" (−0.70), "and" (−0.57) | "health" (+0.35), "and health" (+0.11) |
| "Vitamin C prevents the common cold." | FALSE (0.45) | "the" (−0.69), "the common" (−0.15), "common cold" (−0.13) | "vitamin" (+0.05), "cold" (+0.04) |
| "Wearing a face mask helps reduce the spread of respiratory viruses." | FALSE (0.44) | "the" (−0.48), "reduce" (−0.17), "helps reduce" (−0.14) | "face" (+0.18), "reduce the" (+0.08), "face mask" (+0.05) |
| "Hand washing with soap reduces the spread of infections." | FALSE (0.46) | "the" (−0.61), "reduces" (−0.20), "with" (−0.17) | "washing" (+0.14), "infections" (+0.05) |
| "Garlic is not an effective treatment for high blood | FALSE (0.20) | "is" (−0.52), "an" (−0.52), "effective" (−0.30) | "high" (+0.05), "for" (+0.04), "is not" (+0.03) |

| | | | |
|---|---|---|---|
| pressure." | | | |
| "Is it safe to take ibuprofen during pregnancy?" | FALSE (0.48) | "is" (−0.62), "it" (−0.32), "during" (−0.07) | "pregnancy" (+0.16), "safe to" (+0.07), "during pregnancy" (+0.05) |
| "xyzzy quux blorp" (no recognised words) | TRUE (0.67) | none | none |

Several of these results are informative, even though I do not know whether the claims are true. I describe what the model does, not whether it is right.

### 11.1 Near ties

The claim "Drinking water cures every disease." is labelled TRUE with a probability of 0.52. That is effectively a coin toss. The words that carry the meaning of the claim, "water" and "cures", barely move the score. The decision is determined by a balance between "drinking" and "every", which push toward False, and a handful of small positive contributions. A model with a clear sense of the claim would be far more confident in either direction. The near tie is therefore a warning sign, and it is why the website shows the confidence next to the label.

The example also shows that the model has no notion of medical plausibility. The words that would make the claim obviously false, such as "cures every disease", are not separately represented in the features in a way that carries that meaning. The model responds to vocabulary, and the vocabulary here is ambiguous.

### 11.2 Style-driven errors

The sentence "Researchers are studying the relationship between sleep and health." is a neutral, descriptive statement. It does not claim that anything cures or prevents anything. The model labels it FALSE, with a P(True) of 0.31, and most of that decision comes from the function words "the", "are" and "and". The only substantial positive contribution is from "health", and it is outweighed. The model is responding to the grammatical style of the sentence rather than to any claim it makes. This is the clearest illustration of the wording bias identified in Section 10.4.

The same pattern appears in several other probes. "Vitamin C prevents the common cold." is labelled FALSE, with a P(True) of 0.45, and "the" contributes −0.69 to that decision. "Hand washing with soap reduces the spread of infections." is labelled FALSE, with a P(True) of 0.46, and again the word "the" carries most of the negative weight. These are sentences that a reader would expect a health classifier to handle on the basis of their subject, but in each case the function words decide the answer.

### 11.3 Length-driven errors

The longer probe is a clear case of length-driven behaviour. I wrote a single sentence of about fifty words describing a study of exercise and mood, with a hedge about limited evidence. It has 41 matching terms, and the model labels it FALSE, with a P(True) of 0.265. The largest negative contributions come from "the" (−0.89), "is" (−0.60) and "that" (−0.51). The largest positive contributions are much smaller: "study", "trials" and "over" each contribute about +0.15. With so

many function words in the claim, their negative weights add up, and the content words that might support a True label cannot outweigh them.

This is the mechanism behind the length gap in Table 5. Longer claims contain more function words, so the sum of their negative contributions grows, and the content words that carry the claim's meaning have less influence on the final score. The model is not evaluating the claim's content more harshly when it is long. It is adding up more of the same evidence, and that evidence is largely stylistic.

### 11.4 Unrecognised input

The last probe, "xyzzy quux blorp", contains no words from the training vocabulary. It has no matching terms, so the decision comes entirely from the intercept. The model labels it TRUE with a probability of 0.67. This is a consequence of the balanced training data and of the positive intercept, and it is a genuine risk. A user who types gibberish, or a claim in another language, will receive a TRUE label with moderate confidence. The website displays a note in this case, so that the user is told the answer comes from the baseline. A better system would refuse to classify inputs with no recognised content.

### 11.5 Summary of failure patterns

The failures fall into four groups. The model is near-indifferent on claims whose key words are ambiguous. It is driven by function words on neutral or stylistically plain sentences. It is increasingly dominated by those same function words as claims get longer. And it defaults to True on input it does not recognise. Each of these would be visible in an aggregate accuracy figure only if someone looked for it. The subgroup audit and the explanations are what make them visible.

### 11.6 Why probe examples are useful

Probe examples are not a measure of accuracy. A small number of hand-written claims cannot establish how often the model fails. What they can do is show mechanisms. A probe that shows the intercept dominating, or a function word dominating, demonstrates that the mechanism exists, and it tells the reader what to look for in the full test set. In that sense the probes are a way of generating hypotheses that a larger, labelled test would then confirm or reject. The future work in Section 14 proposes exactly that follow-up.

Probes also serve a second purpose. They show a reader how the system behaves on inputs that do not look like training data, such as nonsense text, a question, or a long paragraph with a hedge. These inputs are common in real use, and an honest report should say how the model treats them.

## 12. Ethical Considerations and Limitations

A classifier for health information carries responsibilities that a classifier for, say, film reviews does not. A label can change what a person believes about a treatment, a vaccine or a disease. This section sets out the concerns that follow from that, and the limits of what this project can claim.

### 12.1 The labels are not medical truth

The labels in PUBHEALTH come from fact-checkers. They reflect a judgement about the claim made by people working with the evidence available at the time. The model learns to predict those

judgements. It does not learn medicine, and it cannot check a claim against current research or clinical guidance. A user who reads the label as a verdict on medical truth is misreading the system, and the website is written to prevent that reading, but the risk remains.

### 12.2 Wording bias

The model relies partly on common function words, so a claim can receive a different label because of how it is phrased. This creates a form of bias that may affect some sources or writing styles more than others. If fact-checkers from one organisation write in a more formal style than those from another, the model could learn to associate that style with a verdict. The explanations make this visible, but they do not remove it. Addressing it properly would require removing stop words, testing for style effects across sources, or training with data that is balanced by source. None of these is done here.

### 12.3 Long claims are missed

Only about one in five verified long claims is labelled True. A person who relied on the model for longer claims would receive many incorrect False labels. Long claims are common in real health communication, because they often include context, caveats and sources. A system that handles them poorly is not fit for those uses, and the results should be read as a warning rather than a finding to be celebrated.

### 12.4 Scope of the dataset

The dataset is in English and contains a substantial amount of COVID-19 content. The model's topical weights reflect that period and the sources in the dataset. Mixture and unproven claims are not covered, although they are common in real life and are often the claims that most need careful judgement. The model cannot say "we do not know", because it was never trained to say so. Its output is always True or False.

### 12.5 Confidence is not accuracy

The confidence figure is the model's probability for the predicted class. It is not a calibrated estimate of how often the model is right. In two of the probe claims, the confidence was between 52% and 56%. The drinking-water claim is labelled TRUE at 52%, which is almost certainly wrong, so a confidence of about half is no guarantee. A reader might take a confidence of 55% as a sign of moderate reliability, when it is closer to a coin toss. The website shows the number, but it does not explain how little it means, and this is a limitation of the interface.

### 12.6 Limited evaluation

The results come from a single train/test split, with no cross-validation and no repeated runs. The decision threshold of 0.5 was not tuned, and the validation split was not used. The long-claim group contains 270 test claims, so its estimates are less precise than those for short claims. A stronger evaluation would address all of these points, and the report should be read as a preliminary audit rather than a definitive measurement.

### 12.7 Privacy and data use

The website does not store the claims that users enter. The server does not write them to disk or log them, and the page does not send them to any third party. The dataset itself is public, but its licence is

not stated, so it is used here only for academic coursework and is not redistributed in the repository. The repository records where to obtain the data.

### 12.8 Intended use and misuse

The application is an educational prototype. It should not be used to make decisions about medical care, to moderate content at scale, or to label people's statements publicly. Using it for any of those purposes would require a much stronger evaluation, a clear statement of its error rates for the relevant population, human oversight, and a process for appealing or correcting its labels. None of these exist in this project. The model card in Annexure A states the intended and out-of-scope uses in the same terms.

### 12.9 Fairness across groups

Fairness in classification usually means comparing error rates across groups of people or of content. This project compares short and long claims, which is a content property, not a group of people. The result nevertheless shows that error rates can differ sharply between groups defined by a simple property of the text. The same approach could be applied to any other grouping, such as the source of the claim, its topic or the language used, and I would expect similar disparities to appear. This is a further reason to audit before deployment.

### 12.10 Responsible-use checklist

Table 7 sets out the steps that would be needed before the system could be used beyond a classroom. Each item is marked with its present status, so that a reader can see what has been done and what remains.

**Table 7: Responsible-use checklist**

| Requirement | Status | Comment |
|---|---|---|
| Statement of intended use | Done | In the model card and on the website |
| Subgroup performance reported | Done | Length audit in Section 10.3 |
| Explanations exposed to users | Done | Why this result panel |
| Warning against medical reliance | Done | Shown with every result |
| Confidence calibration | Not done | Reported probabilities are not calibrated |
| Cross-validation and intervals | Not done | Single split only |
| Stop-word and style analysis | Not done | Required to address wording bias |
| Handling of unrecognised input | Partly done | Note shown; no refusal |
| Human review process | Not done | Needed before any real use |
| Data licence confirmed | Not done | Academic use only; licence not stated |

### 12.11 Alternatives to the current design

It is worth considering what a more cautious design would look like. One option is to make the system abstain. Instead of returning True or False for every input, it could return "insufficient evidence" when the confidence is below a chosen threshold, or when none of the input words are recognised. This would remove the worst behaviour seen in the probes, at the cost of answering fewer claims. Given that the model is least reliable on exactly the inputs where it is least confident, abstention is a reasonable direction to test.

A second option is to present the model only as a ranking tool. The system could sort claims by their probability of being false, so that a human reviewer sees the most suspicious claims first, rather than

receiving a binary label for each claim. This keeps the human in the decision, and it uses the model where it is most useful: prioritising attention, not delivering verdicts.

A third option is to show evidence alongside the label. Retrieving the fact-checker's explanation for each claim, where one exists, would change the output from a prediction into a pointer to a reasoned judgement. This would be a larger change, and it would need careful engineering to avoid presenting an explanation that does not match the claim. It is nevertheless the direction in which the project would most naturally grow.

## 13. Conclusion

This project built HealthGuard, a TF-IDF and Logistic Regression classifier for health claims, trained on the PUBHEALTH fact-checking dataset. The model reaches 76.3% accuracy and a macro-F1 of 0.757 on 987 held-out claims. It is connected to a working website that shows a label, a confidence score and an explanation of the words behind each decision.

The most important finding is that overall accuracy hides a serious weakness. Accuracy falls to 67.0% on long claims, and only 19.8% of long True claims are labelled True. The explanations show why. The model partly learns the style of a claim, not only its topic, because common function words carry some of the largest weights in the model. A reader who looked only at the headline accuracy would have a misleading picture of how the model behaves.

All four objectives were met. The pipeline was built and evaluated, which covers Objective 1. The length audit was carried out and reported in full, which covers Objective 2. The coefficients were extracted and used to explain individual predictions, covering Objective 3. A reproducible notebook, a saved model and a working website were delivered, covering Objective 4. The research questions were answered in Section 10.6.

Looking back, the most useful decision was to report the audit even though it made the model look weaker. The project would have been easier to present with accuracy alone, but that would have implied that the model works equally well for all claims, which the results do not support. The simpler model was also a good choice. Its explanations were exact, and that made it possible to say where the model fails and why.

Overall, the project shows that a transparent, classical model can support a meaningful bias and explainability analysis, and that such an analysis is essential before any classifier of this kind is trusted with health information. It does not show that the model is ready to be used.

### 13.1 Lessons learned

The most useful lesson from the project is that headline metrics can be misleading in a precise and predictable way. Accuracy looked acceptable, and macro-F1 and recall were needed to show what was happening. I would now report a subgroup audit as a first step in any classification project, rather than as an afterthought, because it is the step most likely to reveal a problem that matters to people.

A second lesson concerns the value of transparency. The linear model was a deliberate choice, and at first it seemed to limit the project, because a more powerful model might have scored higher. In the end, the transparency was what made the project useful. The explanations revealed a dependence on

function words that a black-box model might have hidden, and the exact contributions made it possible to show a reader why a particular label was produced.

A third lesson is that the interface matters as much as the model. Showing a label alone would have encouraged the reading that the system gives verdicts. Showing the confidence and the explanation, together with a clear warning, makes the output easier to interpret correctly. The website was the part of the project that most changed how I thought about the users of the system.

A fourth lesson concerns the scope of a student project. It is tempting to try to solve the whole problem, which would mean retrieving evidence, handling mixed claims and building a stronger model. The better course, which this project took, is to choose a narrow question, answer it carefully, and state clearly what remains open. A narrow report with honest limits is more valuable than a broad one whose claims cannot be checked.

## 14. Future Scope

The project leaves several clear directions for further work. I list them in roughly the order I would pursue them, starting with the cheapest and most informative changes.

- **Measure the effect of stop words.** Remove common function words and repeat the length audit. This would show how much of the length gap is caused by wording, and it would clarify whether the explanations are describing a real dependence or an artefact of the features.
- **Tune the decision threshold for long claims.** The 0.5 threshold was never tested. Choosing a threshold separately for long claims, or training with more long-claim examples, might improve recall on the True class. Any change would need to be audited again.
- **Add cross-validation and confidence intervals.** Repeated splits would show how stable the results are, and intervals would make the long-claim estimate more precise.
- **Compare with a contextual model.** A fine-tuned transformer such as BERT [10] could be evaluated with the same length audit. The comparison would show whether the length gap is specific to TF-IDF or is a general problem with this kind of data, and it would need a separate explanation method, such as LIME [11].
- **Add a third outcome.** Mixture and unproven claims could be given their own label, so that the model can express uncertainty rather than forcing a True or False answer. This would require a different dataset arrangement and a different evaluation.
- **Retrieve the evidence.** Link each claim to the fact-checker's explanation, or to the source articles that the dataset already contains. Users could then check the reasoning rather than rely on the label, which would address the main limitation of the current system.
- **Handle unrecognised input.** Refuse to classify claims with no recognised vocabulary, or ask the user to rephrase them, instead of returning a label based only on the baseline.
- **Test with users.** A small study with people who read health claims would show whether the explanations help them judge the model's answers, or whether they are simply trusted as they are.

### 14.1 Priorities

If I had one more month, I would do three things first. The first is the stop-word experiment, because it is cheap, it directly tests the main explanation in the project, and it would show whether the length

gap is caused by wording. The second is cross-validation with confidence intervals, because the long-claim estimate is currently based on 270 claims and one split. The third is a refusal rule for unrecognised input, because it is a small change that removes a clear risk from the interface.

Larger projects would follow. Evidence retrieval would connect each claim to its fact-checker explanation, so that the reasoning could be shown rather than inferred. A contextual model would test whether the weaknesses are specific to TF-IDF. And a user study would show whether the explanations help people judge the output or simply make them more confident in it, which is a separate and important question.

## 15. References


[1] J. Kotonya and F. Toni, "Explainable automated fact-checking for public health claims," in Proc. 2020 Conf. Empirical Methods in Natural Language Processing (EMNLP), 2020.

[2] Hugging Face, "ImperialCollegeLondon/health_fact," Datasets hub. [Online]. Available: https://huggingface.co/datasets/ImperialCollegeLondon/health_fact

[3] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825–2830, 2011.

[4] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," Inf. Process. Manage., vol. 24, no. 5, pp. 513–523, 1988.

[5] Google, "Google Colaboratory." [Online]. Available: https://colab.research.google.com

[6] Pallets Projects, "Flask documentation." [Online]. Available: https://flask.palletsprojects.com

[7] scikit-learn developers, "TfidfVectorizer" and "LogisticRegression," scikit-learn documentation. [Online]. Available: https://scikit-learn.org

[8] A. Vlachos and S. Riedel, "Fact checking: Task definition and dataset construction," in Proc. ACL Workshop on Language Technologies and Computational Social Science, 2014.

[9] J. Thorne, A. Vlachos, C. Christodoulopoulos and A. Mittal, "FEVER: A large-scale dataset for fact extraction and VERification," in Proc. 2018 Conf. North American Chapter of the Association for Computational Linguistics (NAACL-HLT), 2018.

[10] J. Devlin, M.-W. Chang, K. Lee and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in Proc. 2019 Conf. North American Chapter of the Association for Computational Linguistics (NAACL-HLT), 2019.

[11] M. T. Ribeiro, S. Singh and C. Guestrin, "'Why should I trust you?': Explaining the predictions of any classifier," in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, 2016.

[12] J. Buolamwini and T. Gebru, "Gender shades: Intersectional accuracy disparities in commercial gender classification," in Proc. Conf. Fairness, Accountability and Transparency (FAT*), 2018.

[13] M. Mitchell et al., "Model cards for model reporting," in Proc. Conf. Fairness, Accountability and Transparency (FAT*), 2019.

[14] C. D. Manning, P. Raghavan and H. Schütze, Introduction to Information Retrieval. Cambridge, U.K.: Cambridge Univ. Press, 2008.


## 16. GitHub Repository

**GitHub Repository Link:** [https://github.com/anshikachoubeybba2025-creator/HealthGuard]

The repository holds the Colab notebook, the trained model file, the Flask server, the website files (index.html and the background image), the requirements file, the launcher for Windows, and a README that explains the dataset source and the setup steps. The PUBHEALTH data is not stored in the repository; the README gives the download location instead.

Anyone who wants to reproduce the results should run the notebook from the top with the PUBHEALTH files available, and then run the server. The model file in the repository was produced by the notebook in its final state, and the coefficients reported in Section 10 come from that file.

## 17. Team Contribution

| Team Member | PRN | Specific Contribution |
|---|---|---|
| [Your Name] | [Your PRN] | Individual project. Problem selection and dataset checks; notebook implementation (preprocessing, TF-IDF, Logistic Regression, evaluation, length audit and explanations); website design and Flask integration; model card; report and presentation. |

## 18. Annexure (if required)

The model card below summarises the classifier in a standard format [13]. It states what the model is, what it should and should not be used for, how it was evaluated, and where it is weak. The figures are the same as those reported in Section 10.

## Annexure A: Model Card — HealthGuard Health Claim Classifier

# Model Card — HealthGuard Health Claim Classifier

### 1. Model details

| Field | Description |
|---|---|
| Model name | HealthGuard-v1 |
| Task | Binary classification of health claims: True (verified) vs False (misinformation) |
| Model | TF-IDF (unigrams + bigrams, terms in fewer than 2 claims dropped) + Logistic Regression (max_iter=1000, balanced class weights) |
| Training data | PUBHEALTH train split, True and False labels only: 8,079 claims (5,078 True, 3,001 False). Mixture, unproven and 28 malformed rows excluded |
| Test data | PUBHEALTH test split, True and False labels only: 987 claims (599 True, 388 False) |
| Preprocessing | None beyond TF-IDF defaults. No stop-word removal and no negation handling |
| Libraries | Python 3 (Google Colab), scikit-learn 1.6.1, pandas, NumPy, matplotlib; Flask and joblib for the website server |
| Developed by | [Your Name], PRN [Your PRN], SAII — NLP course, October 2026 |

### 2. Intended use

- **Intended:** An educational demonstration of a classical NLP pipeline, and a transparent first check that flags English health claims for human review.
- **Out of scope:** Medical or treatment decisions; claims in other languages; social media posts; mixture or unproven claims; events after the dataset was collected. The model is not a fact-checker, and its output is not a medical verdict.

### 3. Overall performance

The model reaches 76.3% accuracy and 0.757 macro-F1 on the 987-claim test set, beating a majority-class baseline (60.7%) by 15.6 points.

| Model | Accuracy | Precision | Recall | Macro-F1 |
|---|---|---|---|---|
| Majority-class baseline (always True) | 60.7% | — | 100% | 0.378 |
| HealthGuard (TF-IDF + LR) | 76.3% | 0.837 | 0.756 | 0.757 |

Precision and recall are for the True (verified) class.

## 4. Performance across text subgroups

**Key audit finding:** accuracy drops by 12.8 points on long claims (more than 15 words) compared with short claims, and recall for verified long claims is only 19.8%, so most of them are labelled False.

| Subgroup | How it was defined | Test size (n) | Accuracy | Macro-F1 |
|---|---|---|---|---|
| Short claims | 15 words or fewer | 717 | 79.8% | 0.754 |
| Long claims | More than 15 words | 270 | 67.0% | 0.526 |

The model is least reliable on long claims, which is where most verified claims are missed. Accuracy alone hides this; macro-F1 and recall expose it.

## 5. Explainability check

**Method 1 — Top features (Logistic Regression coefficients).** Each word or phrase has a weight. Positive weights push a claim toward True; negative weights push it toward False. Topic words such as "coronavirus", "officials", "study" and "health" lean True, while function words such as "the", "is", "was" and "and" lean False, so the model partly learns writing style.

| Top positive features (toward True) | Top negative features (toward False) |
|---|---|
| coronavirus ([insert weight]) | the ([insert weight]) |
| officials ([insert weight]) | is ([insert weight]) |
| study ([insert weight]) | was ([insert weight]) |
| health ([insert weight]) | and ([insert weight]) |
| mental health ([insert weight]) | that ([insert weight]) |

**Method 2 — [Insert second explainability method and its result, e.g. LIME on one misclassified claim, or the word contributions for one claim].**

## 6. Limitations and ethical considerations

- **Main limitation:** The model does not understand word order or context. It relies on function words and misses many verified long claims (19.8% recall), so it should not be trusted on those without human review.
- **Data bias:** Claims come from fact-checking sites and include COVID-19 content, so the topic words the model learns reflect that period and those sources.
- **Label source:** Labels are fact-checker judgements, not clinical verification. Mixture and unproven claims were excluded, so the model cannot handle mixed or unverified claims.

- **Small audit sample:** The long-claim subgroup has 270 test claims, so its estimate is less precise than the short-claim one. The results also come from one train/test split with no cross-validation, and the 0.5 decision threshold is not tuned.

## 7. Recommendations

- Test removing common function words and compare the length audit again.
- Tune the decision threshold or add more long-claim training data, then re-run the same subgroup audit.
- Add cross-validation to check that the results are stable.
- Compare against a contextual model (for example, a fine-tuned transformer) using the same subgroup audit.
- Route low-confidence predictions to a human reviewer.